ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO ĐỒ ÁN CUỐI KỲ

ĐỀ TÀI

VƯỜN HOA AI

NHẬN DIỆN, TRA CỨU VÀ KHÁM PHÁ 103 LOÀI HOA

Giảng viên hướng dẫn:	Giảng viên phụ trách
Sinh viên thực hiện:	Nguyễn Hữu Lê Hiếu — 23010895 Phạm Văn Huấn — 23010331 Đào Duy Đông — 24100085
Lĩnh vực liên ngành:	Công nghệ thông tin — Trí tuệ nhân tạo

Hà Nội, tháng 10 năm 2026

DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG NHIỆM VỤ

Nhóm thực hiện đồ án Vườn Hoa AI gồm ba sinh viên. Các nhiệm vụ được phân chia theo phần việc chính; nhóm cùng phối hợp tích hợp, rà soát yêu cầu và hoàn thiện sản phẩm. Tỷ lệ đóng góp do nhóm tự đánh giá và cần được từng thành viên xác nhận trên bảng phân công riêng.

STT	Họ và tên	Mã sinh viên	Nhiệm vụ thực hiện	Đóng góp
1	Nguyễn Hữu Lê Hiếu	23010895	Trưởng nhóm; tích hợp sản phẩm, kiến trúc, API/AI, điều phối kiểm thử	34%
2	Phạm Văn Huấn	23010331	Thành viên; giao diện, trải nghiệm người dùng và nội dung catalog	33%
3	Đào Duy Đông	24100085	Thành viên; tài khoản, dữ liệu, kiểm thử và hỗ trợ triển khai	33%

Tổng tỷ lệ đóng góp: 100%.

LỜI CAM ĐOAN

Nhóm sinh viên thực hiện xin cam đoan báo cáo đồ án "Vườn Hoa AI — Nhận diện, tra cứu và khám phá 103 loài hoa" phản ánh quá trình xây dựng sản phẩm của nhóm. Mô tả kiến trúc, API, dữ liệu, kiểm thử và kết quả trong báo cáo được đối chiếu với mã nguồn và các bằng chứng được nêu trong repo tại thời điểm hoàn thiện.

Các tài liệu, thư viện, mô hình và nguồn dữ liệu của bên thứ ba được ghi nhận trong mã nguồn hoặc phần nguồn kiểm chứng. Những giới hạn chưa được kiểm tra đủ rộng được nêu rõ; nhóm không trình bày một lượt smoke test như bằng chứng cho độ chính xác tổng quát.

Nhóm chịu trách nhiệm về nội dung báo cáo và sẽ bổ sung, chỉnh sửa theo góp ý của giảng viên.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Hữu Lê Hiếu · Phạm Văn Huấn · Đào Duy Đông

LỜI CẢM ƠN

Nhóm em xin cảm ơn giảng viên môn Lập trình Web nâng cao đã cung cấp yêu cầu, định hướng và các tiêu chí giúp nhóm xác định phạm vi cho đồ án Vườn Hoa AI. Những nội dung về kiến trúc, API, cơ sở dữ liệu, kiểm thử, triển khai và hỏi đáp cá nhân đã giúp nhóm nhìn sản phẩm dưới góc độ một hệ thống hoàn chỉnh thay vì chỉ một giao diện trình diễn.

Nhóm cũng cảm ơn các thành viên đã cùng trao đổi, xây dựng, kiểm thử và hoàn thiện sản phẩm. Việc tích hợp dữ liệu catalog, giao diện, dịch vụ mô hình, xác thực và kho dữ liệu đòi hỏi thống nhất hợp đồng giữa nhiều phần. Các lần chạy thử và rà soát lỗi đã giúp nhóm điều chỉnh luồng người dùng cũng như ghi nhận rõ hơn những giới hạn còn tồn tại.

Báo cáo và phần mềm vẫn còn những hạn chế về độ lớn của tập đánh giá, phạm vi nhãn detector, chất lượng truy xuất tiếng Việt và khả năng vận hành trên gói tài nguyên miễn phí. Nhóm mong nhận được góp ý để hiểu rõ hơn cách đánh giá một hệ thống AI tích hợp với ứng dụng web và tiếp tục hoàn thiện các phần chưa đạt mức sản phẩm thực tế.

Nhóm xin chân thành cảm ơn.

MỤC LỤC

MỞ ĐẦU **1**
DANH MỤC TỪ VIẾT TẮT 2
Tính cấp thiết của đề tài 1
Mục tiêu nghiên cứu 1
Đối tượng và phạm vi nghiên cứu 1
Phương pháp thực hiện 1
Bố cục báo cáo 1
CHƯƠNG 1. BỐI CẢNH VÀ BÀI TOÁN **3**
1.1. Nhu cầu người dùng 3
1.2. Phát biểu bài toán và tiêu chí thành công 3
1.3. Phạm vi đã triển khai 3
1.4. Quy trình phát triển và cách tổ chức mã 4
CHƯƠNG 2. YÊU CẦU, PHẠM VI VÀ NGƯỜI DÙNG **5**
2.1. Ma trận yêu cầu với phần triển khai 5
2.2. Vai trò và hành trình chính 5
2.3. Yêu cầu phi chức năng 6
2.4. Tiêu chí nghiệm thu cho từng luồng 6
CHƯƠNG 3. KIẾN TRÚC VÀ LUỒNG DỮ LIỆU **7**
3.1. Sơ đồ kiến trúc triển khai 7
3.2. Luồng phân loại từ trình duyệt tới kết quả 7
3.3. Luồng tìm kiếm bằng văn bản và ảnh 8
3.4. Luồng tài khoản và ranh giới tin cậy 8
3.5. Phân loại dữ liệu tĩnh, dữ liệu AI và dữ liệu người dùng 9
CHƯƠNG 4. PHÂN TÍCH GIAO DIỆN REACT **10**
4.1. Thành phần chức năng và React Hooks 10
4.2. Điều hướng và trang bách khoa 10
4.3. API client, timeout và chuẩn hóa lỗi 10
4.4. Tối ưu upload và khả năng tương tác 11
4.5. Trạng thái React và đồng bộ dữ liệu cá nhân 11
CHƯƠNG 5. API VÀ XỬ LÝ YÊU CẦU **12**
5.1. Danh mục REST API chính 12
5.2. Hợp đồng dữ liệu cho yêu cầu ảnh 12
5.3. Streaming chat qua SSE 13
5.4. Validation và phản hồi lỗi 13

CHƯƠNG 6. BỐN CHỨC NĂNG AI 14
6.1. AI 1 — Phân loại ảnh bằng ResNet-18 14
6.2. Dữ liệu huấn luyện và đánh giá classifier 14
6.3. Grad-CAM và diễn giải kết quả 15
6.4. AI 2 — Phát hiện vùng hoa bằng YOLO11n 15
6.5. AI 3 — Tìm ảnh bằng ảnh truy vấn 16
6.6. AI 3 — Tìm kiếm ảnh từ mô tả tiếng Việt 16
6.7. AI 4 — Chatbot RAG và truy xuất nguồn 17
6.8. Phương án dự phòng của chatbot và chi phí 17
6.9. Quản lý RAM và nạp model theo nhu cầu 17
CHƯƠNG 7. TÀI KHOẢN, XÁC THỰC VÀ POSTGRESQL 19
7.1. Mô hình dữ liệu 19
7.2. Đăng ký, đăng nhập và băm mật khẩu 19
7.3. Cookie phiên và vòng đời token 20
7.4. Đọc/ghi payload và cách ly dữ liệu 20
CHƯƠNG 8. CÁC CHỨC NĂNG SẢN PHẨM BỔ TRỢ 21
8.1. Bách khoa 103 loài hoa 21
8.2. Cây của tôi và lịch chăm sóc 21
8.3. Sổ hoa và lịch sử nhận diện 21
8.4. Gợi ý bó hoa: quy tắc thay vì AI 22
8.5. Thiệp chia sẻ và nội dung lời chúc 22
CHƯƠNG 9. KIỂM THỬ, CI VÀ CHẤT LƯỢNG 23
9.1. Unit test và kiểm thử hợp đồng API 23
9.2. Kiểm thử xác thực và PostgreSQL 23
9.3. Kiểm thử frontend và trạng thái lỗi 23
9.4. CI, build và quét bảo mật 24
9.5. Phân tầng bằng chứng kiểm thử 24
CHƯƠNG 10. TRIỂN KHAI VÀ VẬN HÀNH 25
10.1. Phát triển cục bộ bằng Docker Compose 25
10.2. Frontend Vercel và API Render 25
10.3. Readiness, health và quan sát lỗi 25
10.4. Kịch bản vận hành khi demo 26
CHƯƠNG 11. ĐÁNH GIÁ, GIỚI HẠN VÀ HƯỚNG PHÁT TRIỂN 27
11.1. Kết quả benchmark tổng hợp 27
11.2. Phân tích lỗi và nguyên nhân có thể 27
11.3. Rủi ro, giới hạn và trạng thái cần theo dõi 28

11.4. Lộ trình cải tiến ưu tiên 28
CHƯƠNG 12. HƯỚNG DẪN ĐỌC CODE VÀ KỊCH BẢN BẢO VỆ 29
12.1. Đường đi từ UI tới API 29
12.2. Đường đi từ login tới bản ghi CSDL 29
12.3. Đường đi retrieval và chatbot 29
12.4. Bộ câu hỏi tự kiểm tra trước khi bảo vệ 30
12.5. Phân công công việc và trách nhiệm nhóm 30
12.6. Khai báo sử dụng công cụ AI 31
KẾT LUẬN 32
NGUỒN KIỂM CHỨNG TRONG REPOSITORY 33
PHỤ LỤC A. MỘT SỐ ĐOẠN CODE CẦN NẮM 34
A.1. Khóa suy luận và đường nạp model 34
A.2. Token phiên không lưu trực tiếp 34
A.3. Dùng cùng model để phân loại và truy xuất 34
A.4. Kiểm tra payload và cách ly theo user 35
A.5. Tài liệu đánh giá cần giữ kèm 35

MỞ ĐẦU

Tính cấp thiết của đề tài

Người gặp một bông hoa thường phải chuyển qua nhiều nguồn để tìm tên, xem ảnh đối chiếu, đọc thông tin chăm sóc và tìm ý nghĩa làm quà. Vườn Hoa AI gom các bước tra cứu này vào một ứng dụng web tiếng Việt, cho phép bắt đầu bằng ảnh, từ khóa hoặc danh mục. Đề tài cũng tạo bối cảnh thực tế để áp dụng API, xác thực, cơ sở dữ liệu, kiểm thử tự động và tích hợp mô hình AI trong cùng một sản phẩm.

Mục tiêu nghiên cứu

Mục tiêu là xây dựng website tra cứu 103 loài hoa, cung cấp bốn nhóm chức năng AI gồm phân loại ảnh, phát hiện vùng hoa, tìm kiếm ảnh theo ảnh hoặc mô tả, và chatbot RAG có trích nguồn. Sản phẩm còn có bách khoa hoa, tài khoản để lưu dữ liệu cá nhân, cây của tôi và luồng gợi ý tặng hoa. Nhóm đánh giá từng luồng bằng kiểm thử phù hợp và ghi rõ giới hạn của số liệu.

Đối tượng và phạm vi nghiên cứu

Đối tượng sử dụng là người muốn nhận biết hoa, tìm thông tin cơ bản, lưu loài đã xem hoặc ghi nhớ lịch chăm cây. Phạm vi AI giới hạn theo nhãn, dữ liệu và tài liệu hiện có; ứng dụng không nhận diện mọi loài thực vật, không chẩn đoán bệnh cây và không thay thế tư vấn chuyên môn về độc tính hay sử dụng cây.

Phương pháp thực hiện

Nhóm phân tích yêu cầu, chia sản phẩm thành giao diện React, API, dịch vụ suy luận và kho dữ liệu; sau đó triển khai từng luồng, kiểm thử bằng unit/API/integration test và smoke test production. Với AI, nhóm dùng metric riêng theo tác vụ và báo cáo rõ kích thước mẫu, môi trường, phiên bản cùng những điều phép đo chưa chứng minh.

Bố cục báo cáo

Báo cáo lần lượt trình bày bối cảnh và yêu cầu, kiến trúc, giao diện React, REST API, bốn chức năng AI, xác thực và PostgreSQL, các chức năng bổ trợ, kiểm thử/CI, triển khai, đánh giá giới hạn, hướng đọc code và các đoạn mã quan trọng. Các đường dẫn production được ghi riêng trong file link ở thư mục gốc repo.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
API	Giao diện lập trình ứng dụng
REST	Kiểu tổ chức tài nguyên và thao tác qua HTTP
SPA	Ứng dụng web một trang
RAG	Sinh câu trả lời có truy xuất tài liệu
SSE	Server-Sent Events, luồng sự kiện từ máy chủ tới trình duyệt
OOD	Dữ liệu ngoài miền huấn luyện
IoU	Giao của hai vùng chia cho hợp của hai vùng
mAP	Trung bình độ chính xác trung bình của bộ phát hiện
P@5	Tỷ lệ kết quả phù hợp trong năm vị trí đầu
CI	Tích hợp liên tục, tự động chạy kiểm tra khi có thay đổi

CHƯƠNG 1. BỐI CẢNH VÀ BÀI TOÁN

1.1. Nhu cầu người dùng

Người gặp một bông hoa thường phải chuyển qua nhiều nguồn để tìm tên, xem ảnh đối chiếu, đọc thông tin chăm sóc và tìm ý nghĩa làm quà. Các trang tìm kiếm tổng quát có thể trả về nhiều kết quả không thống nhất, còn một danh mục tĩnh không giúp người dùng nhận diện ảnh riêng của mình. Nhóm chọn đề tài vì hoa là chủ đề gần gũi, có dữ liệu ảnh và thông tin để thử nghiệm nhiều dạng tương tác trên cùng một ứng dụng web.

Đối tượng chính là người dùng phổ thông muốn tra cứu nhanh, người trồng cây tại nhà cần ghi nhớ lịch chăm sóc, và người muốn tìm ý tưởng tặng hoa. Họ không nhất thiết biết tên khoa học hay tên tiếng Anh. Vì vậy, ứng dụng cho phép bắt đầu bằng ảnh, từ khóa tiếng Việt hoặc lựa chọn trong danh mục. Kết quả cần giải thích đủ để người dùng biết đây là gợi ý của mô hình hay thông tin được tra từ dữ liệu.

1.2. Phát biểu bài toán và tiêu chí thành công

Bài toán sản phẩm là tạo một điểm vào duy nhất để người dùng đi từ câu hỏi "đây là hoa gì?" tới các bước tiếp theo: xem thông tin loài, tìm ảnh gần giống, hỏi về cách trồng hoặc lưu cây vào tài khoản. Bài toán môn học rộng hơn: chứng minh một giao diện React thực sự gọi API, backend có xác thực và CSDL, kiểm thử tự động, có địa chỉ truy cập Internet và có thể giải thích kiến trúc bằng code.

Nhóm đánh giá thành công theo ba lớp. Lớp chức năng xem xét từng luồng có thể đi từ đầu vào tới kết quả và có trạng thái lỗi. Lớp kỹ thuật xem xét hợp đồng API, dữ liệu, quyền truy cập, cách chạy và kiểm thử. Lớp AI xem xét metric phù hợp với từng tác vụ, đồng thời nêu rõ tập dữ liệu và điều gì phép đo chưa chứng minh. Một lượt chạy thành công không đồng nghĩa mô hình đúng cho mọi ảnh; vì thế báo cáo phân biệt kiểm thử API, smoke test production và đánh giá tập dữ liệu.

1.3. Phạm vi đã triển khai

Phạm vi người dùng gồm trang chủ, bách khoa 103 loài, trang chi tiết, nhận diện ảnh, phát hiện đối tượng trong ảnh hoặc video/webcam, tìm ảnh bằng mô tả hoặc ảnh mẫu, chatbot về hoa, sổ hoa, cây của tôi và gợi ý tặng hoa. Một số dữ liệu danh mục và

ảnh được đóng gói cùng ứng dụng để trang tra cứu hoạt động nhanh và có thể chạy ngoại tuyến trong một số luồng.

Phạm vi AI tập trung vào nhận diện loài theo tập nhãn có sẵn, phát hiện một số nhóm hoa, xếp hạng hình ảnh và trả lời dựa trên kho tài liệu. Hệ thống không xác định mọi loại thực vật, không chẩn đoán bệnh cây, không đưa ra hướng dẫn ăn/uống hoặc độc tính. Việc đưa ra ranh giới này giúp giao diện không biến một điểm số mô hình thành lời khuyên chuyên môn có hậu quả cao.

1.4. Quy trình phát triển và cách tổ chức mã

Repository được chia thành phần giao diện trong web/src, API trong api, mô hình và logic truy xuất trong core, dữ liệu mô tả ở data, kiểm thử ở tests và bằng chứng/metric ở reports. Cách chia này tạo được ranh giới đọc code: thành phần React lo tương tác; route API chuyển request; lớp nghiệp vụ mô hình lo suy luận; kho tài khoản lưu dữ liệu bền vững. Một số chức năng phía trình duyệt như gợi ý bó hoa có thể dùng trực tiếp dữ liệu tĩnh và không cần chờ API.

Việc nhóm thay đổi trên nhiều nhánh và cấu hình triển khai có thể làm mã production khác mã trong checkout. Vì vậy, báo cáo phân biệt kết quả trên phiên bản đã ghi trong bằng chứng production và mô tả code hiện thấy trong repository. Mỗi thành viên cần đọc phần mình phụ trách, chạy lại demo trên máy trình bày và đối chiếu SHA/nhánh trước khi bảo vệ; tài liệu này không thay cho việc kiểm tra trạng thái deploy ngay tại ngày trình bày.

CHƯƠNG 2. YÊU CẦU, PHẠM VI VÀ NGƯỜI DÙNG

2.1. Ma trận yêu cầu với phần triển khai

Yêu cầu môn học	Phần tương ứng	Bằng chứng kiểm tra
React function components, Hooks, gọi API thật	web/src; các trang AI gọi api.js	React build, browser Network và component tests
Backend và REST API	FastAPI trong api/main.py; Express gateway cho tài khoản	OpenAPI, API tests, health endpoint
Xác thực và CSDL	cookie phiên, api/account_store.py, SQLite local/PostgreSQL production	auth integration tests, account smoke evidence
Kiểm thử tự động	tests/ và web/src/*.test.js	GitHub Actions workflow
Deploy Internet	Vercel frontend, Render API, Neon PostgreSQL	production URLs và dated smoke records
Điểm cộng AI và Docker/CI	bốn tác vụ AI, Compose, CI	model evaluation reports và compose smoke

Ma trận trên liên kết yêu cầu với nơi kiểm tra thay vì chỉ nêu tên công nghệ. Để xác nhận một tiêu chí, cần xem cả mã nguồn, kiểm thử tương ứng và môi trường chạy. Chẳng hạn, tệp compose hợp lệ chứng minh khai báo dịch vụ có cú pháp đúng, nhưng chỉ smoke test sau khi dựng mới chứng minh container khởi chạy được. Tương tự, route đăng nhập có trong code không đủ để chứng minh một bản deploy cụ thể đã cập nhật route đó.

2.2. Vai trò và hành trình chính

Người chưa đăng nhập vẫn được tra cứu danh mục và dùng các chức năng khám phá AI công khai; tài khoản cần thiết khi muốn đồng bộ dữ liệu cá nhân. Hành trình nhận diện bắt đầu từ chọn ảnh hoặc ảnh mẫu, hiển thị trạng thái đang xử lý, trả top-k nhãn, rồi dẫn tới trang bách khoa hoặc lưu loài yêu thích. Hành trình tìm kiếm bắt đầu từ mô tả tiếng Việt hoặc ảnh truy vấn, sau đó xếp ảnh trong thư viện theo điểm tương đồng.

Hành trình chăm sóc bắt đầu khi người dùng thêm một loài vào Cây của tôi, cấu hình loại việc và khoảng ngày, hoàn thành việc chăm sóc rồi nhận lịch tiếp theo. Sổ hoa ghi nhớ loài đã nhận diện và mục yêu thích. Khi người dùng đăng nhập, các bộ sưu tập

được đọc từ server và hợp nhất với dữ liệu lưu ở trình duyệt. Hành trình tặng hoa lấy dịp, người nhận, màu và kích cỡ làm điều kiện để tạo các đề xuất; lời chúc có thể dùng mẫu ngoại tuyến hoặc dịch vụ sinh văn bản nếu provider đã bật.

2.3. Yêu cầu phi chức năng

Giao diện phải có trạng thái tải, rỗng, thành công và lỗi để người dùng không hiểu nhầm yêu cầu đang chạy. Đầu vào ảnh cần được kiểm tra ở trình duyệt và backend. Mô hình nặng cần tải theo nhu cầu hoặc dùng chế độ giảm RAM để dịch vụ gói miễn phí không phải giữ mọi trọng số cùng lúc. API phải trả lỗi có cấu trúc, không đưa stack trace hoặc bí mật cấu hình ra phản hồi.

Yêu cầu lưu trữ gồm giữ dữ liệu tài khoản qua lần mở lại, cô lập theo người dùng và xóa phiên khi đăng xuất. Giao diện cần giảm kích thước ảnh trước khi gửi khi trình duyệt hỗ trợ, và tôn trọng cài đặt giảm chuyển động cho animation kết quả. Khả năng vận hành được củng cố bằng health/startup probe, log có thời gian xử lý và cấu hình chạy Compose. Những điểm này giảm sự cố phổ biến nhưng chưa thay thế kiểm thử tải, backup/restore hoặc đánh giá trợ năng đầy đủ.

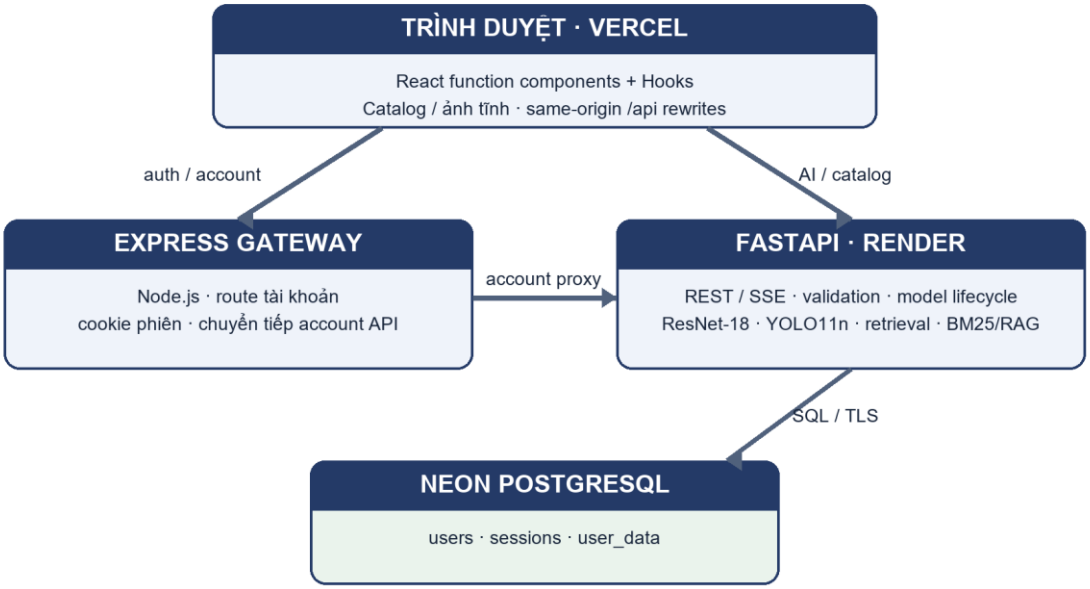
2.4. Tiêu chí nghiệm thu cho từng luồng

Một luồng được coi là có thể trình diễn khi đầu vào hợp lệ tạo kết quả có thể hiểu, đầu vào sai trả thông báo có hướng xử lý, và dữ liệu cá nhân không truy cập được nếu chưa xác thực. Với AI, cần báo metric riêng: accuracy/F1 cho phân loại, mAP cho hộp giới hạn, P@5 cho xếp hạng ảnh, Hit@3 cho truy xuất tài liệu. Không thể gộp các con số thành một "độ chính xác AI" duy nhất vì mỗi tác vụ có mục tiêu và cách đo khác nhau.

Kịch bản nghiệm thu cũng kiểm tra trạng thái triển khai: URL frontend tải được, API trả health, login và account endpoint đi qua cùng origin, các yêu cầu model trả kết quả hoặc báo lỗi hợp lệ. Một lần test có thể chỉ kiểm tra một ảnh mẫu; khi đó báo cáo gọi là smoke test chứ không mô tả như kiểm định đại diện. Ngày chốt báo cáo, các kết quả production được gắn mốc thời gian trong reports để người đọc hiểu giới hạn về độ mới.

CHƯƠNG 3. KIẾN TRÚC VÀ LUỒNG DỮ LIỆU

3.1. Sơ đồ kiến trúc triển khai



Hình 3.1. Luồng định tuyến từ frontend tới dịch vụ và cơ sở dữ liệu

Frontend là lớp trình bày và điều phối tương tác. Các yêu cầu tới API được tập trung trong web/src/api.js để thống nhất timeout, credentials, xử lý lỗi, upload ảnh và stream SSE. Vercel rewrite hỗ trợ dùng cùng origin cho route tài khoản, tránh cookie phiên trở thành cookie bên thứ ba. Express gateway là một ranh giới API cho đăng nhập và dữ liệu; FastAPI đảm nhiệm endpoint AI và, theo cấu hình fallback, xử lý account proxy. Neon lưu các bản ghi tài khoản trên PostgreSQL.

Việc tách API AI với gateway tài khoản phản ánh khác biệt về tải và runtime: mô hình PyTorch/Ultralytics phụ thuộc Python và RAM; gateway tài khoản nhẹ hơn, chạy Node. Dù vậy, tách service làm tăng số cấu hình deploy, log và điểm có thể sai. Cần quản lý rewrite, CORS, cookie và môi trường giữa các service; vì vậy kiến trúc phải được mô tả bằng route thật và không nên gọi mọi module là microservice độc lập nếu chúng chưa được triển khai tách rời.

3.2. Luồng phân loại từ trình duyệt tới kết quả

React component nhận File từ ImagePicker. Hàm postImage gọi prepareImage để tạo ảnh JPEG tối đa cạnh 1024 nếu browser decode được; sau đó gửi multipart/form-data. API FastAPI đọc ảnh, kiểm tra định dạng, chuyển ảnh sang RGB và gọi classifier.

Kết quả predictions, confidence flag và latency được trả dưới JSON. React ánh xạ nhãn thành bản ghi loài, vẽ thanh xác suất và hiện cảnh báo nếu nhãn top-1 dưới ngưỡng.

Nếu người dùng bật Grad-CAM, component gửi explain=true. Server chạy thêm lượt lan truyền ngược và trả ảnh phủ heatmap dạng data URL. Đây là tác vụ nặng hơn inference thường và phần giải thích chỉ cho biết vùng có ảnh hưởng tới điểm của lớp trong mô hình; không chứng minh mô hình hiểu cấu trúc sinh học. Khi kết quả đủ tin cậy, UI mới đưa tùy chọn ghi nhớ loài hoặc xem gợi ý tặng hoa liên quan.

3.3. Luồng tìm kiếm bằng văn bản và ảnh

Với truy vấn văn bản, client gửi câu mô tả, số kết quả và nhãn lọc tùy chọn. Search engine chuẩn hóa tiếng Việt, xác định alias hoặc cue hình thái phù hợp, rồi mã hóa query thành vector để so với thư viện ảnh. Một số truy vấn tiếng Việt không nằm trong không gian ngôn ngữ của encoder production; nhánh fallback dùng tín hiệu từ cụm từ, tên loài và đặc điểm để tăng khả năng tìm đúng nhóm. API trả query_used để UI minh bạch hóa truy vấn được sử dụng.

Với ảnh, service trích vector từ ResNet-18 và tính tích vô hướng với vector thư viện đã chuẩn hóa. Nếu dự đoán loài đủ chắc chắn, danh sách có thể được thu hẹp theo nhãn đó trước khi xếp hạng. Lọc theo lớp giúp kết quả cùng loài hơn trên bộ thử, nhưng phụ thuộc classifier: nếu classifier chọn sai nhãn, truy xuất có thể bỏ qua ảnh phù hợp ở lớp khác. Đây là tradeoff giữa độ chính xác trong cùng lớp và khả năng tìm rộng.

3.4. Luồng tài khoản và ranh giới tin cậy

Browser gửi đăng ký hoặc login qua API cùng origin; server băm mật khẩu để so sánh, tạo token ngẫu nhiên và chỉ lưu SHA-256 của token trong bảng sessions. Cookie chứa token được cấu hình HttpOnly để script phía client không đọc trực tiếp. Mỗi truy vấn /api/account/data/{key} xác minh token, tìm user hiện hành, sau đó truy xuất bản ghi có user_id khớp với phiên.

Ranh giới tin cậy là trình duyệt không được tự quyết định user_id để đọc dữ liệu. Backend lấy user từ cookie phiên rồi truyền ID đó xuống store. Điều này ngăn người dùng sửa URL để đọc bộ sưu tập của người khác, miễn là mọi route đều theo cùng quy tắc. Dữ liệu localStorage vẫn có thể hiện diện trên máy dùng chung, vì vậy logout cần

xóa trạng thái giao diện/phiên đúng cách và người dùng nên tránh lưu dữ liệu nhạy cảm trong Sổ hoa.

3.5. Phân loại dữ liệu tĩnh, dữ liệu AI và dữ liệu người dùng

Danh mục 103 loài, dữ liệu mô tả, ảnh mẫu và ý nghĩa hoa được quản lý như tài nguyên ứng dụng trong data/ và web/public. Chúng thay đổi cùng phiên bản code, không cần ghi lại cho từng tài khoản. Checkpoint, nhãn và index ảnh nằm trong thư mục artifact của mô hình, được nạp khi backend chạy. Dữ liệu của người dùng gồm tài khoản, phiên và payload vườn/sổ hoa nên được lưu trong cơ sở dữ liệu.

Ba nhóm có vòng đời khác nhau. Dữ liệu tĩnh cần kiểm tra tính nhất quán với gallery ảnh và ID của loài. Model artifact cần phiên bản, checksum và nhãn tương thích với code suy luận. Payload tài khoản cần kiểm soát quyền, backup và chính sách retention. Nếu đặt chung tất cả vào một bảng JSON lớn, sẽ khó truy vấn và khó phân tách quyền; nếu biến mọi mô tả loài thành một bản ghi user, sẽ nhân dữ liệu và gây khó nâng cấp. Hiện ứng dụng giữ catalog tĩnh và hai kho dữ liệu cá nhân theo data_key.

CHƯƠNG 4. PHÂN TÍCH GIAO DIỆN REACT

4.1. Thành phần chức năng và React Hooks

Mỗi trang như EncyclopediaPage, FlowerDetailPage, Classify, Detect hay Chat được chia thành component chức năng. useState giữ trạng thái giao diện thay đổi theo thao tác; useEffect đồng bộ những việc phụ như animation, nạp dữ liệu hoặc đồng bộ khi tài khoản đổi; useRef giữ tham chiếu canvas hay media element mà không gây render thừa. Các component nhỏ như ImagePicker giữ trách nhiệm chọn file/preview, còn API client đảm nhiệm network.

Trong Classify.jsx, state được biểu diễn bằng object có status idle/loading/ok/error. Kiểu trạng thái này làm rõ các nhánh UI và tránh một tập biến boolean xung đột nhau, chẳng hạn isLoading cùng lúc với isError. Khi request thất bại, component lưu cả file để nút Thử lại dùng đúng ảnh ban đầu. Sau khi có kết quả, effect tạo animation thanh điểm; cleanup hủy requestAnimationFrame để không cập nhật component đã unmount.

4.2. Điều hướng và trang bách khoa

App.jsx phân giải route bằng location và điều hướng nội bộ đến danh mục, chi tiết loài, khu vườn hay trang chia sẻ. Bách khoa dựa trên data/species.json thay vì gọi một endpoint cho từng lần duyệt. Điều đó giúp bộ lọc và trang chi tiết phản hồi nhanh; API vẫn có endpoint danh mục để client ngoài hoặc ứng dụng khác truy cập dữ liệu chung.

Trang chi tiết nhận một bản ghi loài đã tìm theo id, từ đó hiển thị tên, mô tả, mùa hoa, đặc điểm và ảnh riêng của từng loài. Tính đúng của liên kết phụ thuộc việc id thống nhất giữa catalog, classifier label mapping và file ảnh. Nếu thiếu bản ghi, giao diện cần trạng thái not found thay vì render nửa trang. Các ảnh được lazy load ở khu vực gallery nhằm tránh tải đồng thời toàn bộ bộ sưu tập khi người dùng mới mở trang.

4.3. API client, timeout và chuẩn hóa lỗi

web/src/api.js là một điểm quan trọng để đọc luồng frontend. API_BASE chọn same-origin trên Vercel để cookie tài khoản được gửi đúng ngữ cảnh, còn local development có thể dùng địa chỉ backend theo cấu hình. createDeadline liên kết AbortSignal của caller với timer; khi quá hạn, request bị hủy và được đổi thành TimeoutError thân thiện. Hàm handle đọc JSON lỗi nếu có và tạo thông báo chứa status HTTP.

Tập trung xử lý lỗi giúp tất cả feature phân biệt lỗi mạng, timeout, dữ liệu sai, lỗi 503 do model chưa sẵn sàng và lỗi gateway 502/504. postJson gắn Content-Type và stringify body; postImage không tự đặt Content-Type multipart vì trình duyệt cần tự sinh boundary. credentials include cho phép gửi cookie phiên cùng yêu cầu. Đây là ví dụ cụ thể về một lớp tiện ích ảnh hưởng đồng thời tới mọi luồng tính năng.

4.4. Tối ưu upload và khả năng tương tác

prepareImage thử giải mã ảnh trong browser, giảm kích thước cạnh dài tối đa 1024 pixel rồi xuất JPEG chất lượng 0,84. Cách làm này giảm lượng dữ liệu qua mạng và kích thước tensor đầu vào; nếu API Canvas hoặc giải mã không hỗ trợ, hàm trả file gốc để không chặn người dùng. Server vẫn phải kiểm tra file độc lập vì client không phải ranh giới bảo mật.

Các component có thông báo role=alert hoặc role=status cho lỗi và trạng thái, nhãn aria-label cho nút vùng phát hiện và mô tả ảnh. Nút dùng type=button để không vô tình submit form cha. Cài đặt prefers-reduced-motion được kiểm tra trước khi chạy animation. Các biện pháp này cải thiện khả năng dùng bàn phím, screen reader và thiết bị yếu, nhưng vẫn cần audit trợ năng toàn trang để xác nhận tương phản, thứ tự focus và ngữ nghĩa heading.

4.5. Trạng thái React và đồng bộ dữ liệu cá nhân

Các bộ sưu tập có helper riêng như giftBook.js và gardenPlanner.js. Hàm đọc/ghi local storage giữ trải nghiệm ngay cả trước khi đăng nhập; accountSync.js điều phối phiên người dùng và tải dữ liệu server. Khi có dữ liệu ở cả hai nơi, mergeAccountCollection dùng identity ổn định để tránh tạo bản ghi trùng. Cách hợp nhất ưu tiên bảo toàn những mục cục bộ mới và nhận dạng theo id thay vì phụ thuộc vị trí mảng.

Đồng bộ hai chiều có rủi ro ghi đè nếu hai thiết bị cùng sửa một payload mà không có version hoặc timestamp có quy tắc conflict. Ứng dụng hiện hợp nhất các collection chức năng chứ chưa triển khai CRDT hoặc nhật ký sự kiện. Vì vậy, nhóm cần mô tả chính xác là đồng bộ dữ liệu ứng dụng, không tuyên bố giải quyết mọi xung đột phân tán. Một hướng cải tiến là thêm optimistic concurrency bằng updated_at/version và thông báo khi bản server đã đổi.

CHƯƠNG 5. API VÀ XỬ LÝ YÊU CẦU

5.1. Danh mục REST API chính

Nhóm	Endpoint	Phương thức / đầu vào	Kết quả chính
Sức khỏe	/api/health, /api/startup	GET	trạng thái service, component và readiness
Loài	/api/species, /api/species/{id}/image	GET	catalog hoặc ảnh hợp lệ
Phân loại	/api/classify	POST multipart file, top_k, explain	top-k, confidence, Grad-CAM tùy chọn
Phát hiện	/api/detect	POST multipart file, conf	hộp, nhãn, điểm và ảnh annotate
Truy xuất	/api/search/text, /api/search/image	POST JSON hoặc multipart	danh sách gallery có điểm tương đồng
Chat	/api/chat, /api/chat/sync, /api/ask	POST JSON	SSE hoặc câu trả lời và nguồn
Tài khoản	/api/auth/register, login, logout, me	POST/GET	tài khoản hiện hành và cookie phiên
Dữ liệu cá nhân	/api/account/data/{key}	GET/PUT	payload garden/giftbook theo phiên
Tặng hoa	/api/gifts/recommend, /api/gifts/greeting	POST JSON	bó gợi ý theo luật, lời chúc tùy provider

Các route được đặt dưới /api để phân tách khỏi tài nguyên frontend. FastAPI/Pydantic kiểm tra kiểu và ràng buộc trường trước khi logic mô hình chạy. Lỗi thiếu trường hoặc sai kiểu thường trả 422; dữ liệu trống hoặc không hợp lệ về nghiệp vụ trả 400; tài nguyên không có trả 404; model chưa được bật trả 503. Dùng status có ý nghĩa giúp client quyết định nên sửa input, thử lại hay hiển thị thông báo tạm thời.

5.2. Hợp đồng dữ liệu cho yêu cầu ảnh

Các endpoint nhận ảnh dùng multipart field tên file. Điều này cho phép gửi nhị phân mà không phải base64 hóa ở request. Endpoint classify hỗ trợ top_k và explain; detect nhận confidence threshold; image search nhận k và label tùy chọn. Backend chuyển dữ liệu qua Pillow để xác nhận có thể mở ảnh trước khi cấp cho thư viện AI. Tên file do client gửi không được dùng làm đường dẫn tin cậy.

Phản hồi phân loại chứa danh sách nhãn/score và cờ confident. Phản hồi detect chứa danh sách box_xyxy theo tọa độ pixel ảnh mô hình và summary theo nhãn; client quy đổi tọa độ sang phần trăm trên kích thước ảnh gốc. Phản hồi search trả id, label, score và URL ảnh gallery. Hợp đồng này giảm phụ thuộc giữa UI và chi tiết nội bộ của model, nhưng cần phiên bản hóa nếu đổi quy tắc tọa độ, thang điểm hoặc cấu trúc field.

5.3. Streaming chat qua SSE

EventSource của trình duyệt không hỗ trợ POST với JSON tùy ý, vì vậy api.js dùng fetch, đọc ReadableStream và TextDecoder. Byte nhận được có thể chia giữa chừng một dòng, nên code gom vào buffer, tách tại dấu xuống dòng kép và giữ phần event chưa hoàn chỉnh cho lượt đọc tiếp. Event type sources cập nhật danh sách tài liệu; token nối vào câu trả lời; event done kết thúc trạng thái đang sinh.

Phương thức stream giảm thời gian chờ cảm nhận: người dùng thấy token đầu tiên trước khi toàn câu hoàn tất. Phần khó là xử lý stream bị ngắt, JSON event sai định dạng, tín hiệu abort và lỗi khi server gửi event theo boundary không trùng chunk mạng. Client cần phân biệt "đã nhận một phần" với "đã nhận hoàn tất". Endpoint đồng bộ /api/chat/sync là lựa chọn đơn giản hơn cho smoke test, tích hợp bên ngoài hoặc khi không cần hiệu ứng streaming.

5.4. Validation và phản hồi lỗi

api/main.py đăng ký exception handler cho HTTPException, RequestValidationError và lỗi chưa dự kiến. Handler validation chuyển lỗi thành danh sách type/loc/msg mà không phản chiếu giá trị người dùng nhập. Handler cuối ghi loại exception và path ở log server, còn client chỉ nhận thông báo chung. Cách này giữ đủ dấu vết cho vận hành mà hạn chế rò stack trace, đường dẫn máy chủ hoặc nội dung nhạy cảm.

API cũng kiểm tra tham số truy vấn như limit, k, id loài, nhãn lọc và message. Các test xác nhận trường hợp blank, thiếu trường, sai định dạng, file không mở được, id âm và tài nguyên không tồn tại. Tuy nhiên, schema validation không tự giới hạn tổng byte upload nếu lớp reverse proxy hoặc server chưa đặt giới hạn phù hợp; production cần cùng lúc cấu hình MAX_UPLOAD_MB, proxy body-size và timeout để giảm nguy cơ cạn tài nguyên.

CHƯƠNG 6. BỐN CHỨC NĂNG AI

6.1. AI 1 — Phân loại ảnh bằng ResNet-18

Classifier dùng ResNet-18 khởi tạo từ trọng số ImageNet rồi thay lớp cuối theo số lớp hoa trong classes.json. Pipeline đánh giá chuyển RGB, resize cạnh ngắn, center crop 224x224, đổi tensor và chuẩn hóa theo mean/std ImageNet. Khi huấn luyện, augmentation gồm random resized crop, flip ngang và color jitter để mô hình không chỉ ghi nhớ một góc chụp hay mức sáng. Trên production CPU hạn chế RAM, tham số được đổi sang BF16 khi chế độ low-memory bật; logits/probability được tính lại ở FP32.

Mô hình trả top-k thay vì chỉ một tên để thể hiện sự không chắc chắn. Cờ confident so điểm top-1 với ngưỡng; giao diện cảnh báo ảnh có thể ngoài miền nhãn. Cờ này không phải bộ phát hiện OOD hoàn hảo vì softmax có thể tự tin trên ảnh sai miền. Điểm số nên được đọc như độ phù hợp tương đối trong tập lớp của mô hình, không phải xác suất sinh học đã hiệu chuẩn cho mọi bối cảnh.

```
def predict(self, image, top_k=3):
    x = EVAL_TF(image.convert("RGB")).unsqueeze(0)
    with self._inference_lock, torch.inference_mode():
        probs = self.model(x).float().softmax(dim=-1)[0]
    scores, idx = probs.topk(min(top_k, len(self.classes)))
    return {"predictions": ..., "confident": scores[0] >= self.min_confidence}
```

Đoạn rút gọn thể hiện ba điểm quan trọng: cùng preprocessing cho mọi request, khóa suy luận để tránh đồng thời đụng model shared trên CPU, và trả nhiều nhãn. Source thực tế: core/classifier.py; route HTTP ở api/main.py; giao diện ở web/src/features/Classify.jsx.

6.2. Dữ liệu huấn luyện và đánh giá classifier

Bằng chứng metric ghi nhận 11.859 ảnh thuộc 103 lớp và split phân tầng với random_state=42; tập validation gồm 2.135 ảnh. Accuracy là 90,26%, macro-F1 91,56% và weighted-F1 90,24%. Accuracy cho biết tỷ lệ đúng chung; weighted-F1 phản ánh trọng số theo số mẫu; macro-F1 cho mỗi lớp trọng số ngang nhau nên giúp nhận ra lớp ít ảnh bị bỏ qua. Việc báo cả ba số làm rõ hệ quả mất cân bằng dữ liệu.

Các số này được xem là kết quả validation trong quy trình phát triển, không phải benchmark test độc lập. Checkpoint có thể đã được chọn dựa trên validation, làm metric

lạc quan hơn khi ước lượng khả năng tổng quát. Để nâng chất lượng khoa học, cần một test split giữ kín tới cuối, kiểm tra ảnh trùng hoặc gần trùng giữa các split, công bố phân bố số mẫu từng lớp và đánh giá ảnh chụp ngoài bộ dữ liệu với điều kiện ánh sáng, nền và thiết bị khác nhau.

6.3. Grad-CAM và diễn giải kết quả

Tính năng explain đăng forward hook tại lớp cuối của layer4 để lưu activation và gradient. Với lớp được chọn, code tính trung bình gradient theo chiều không gian làm trọng số kênh, nhân với activation, cộng các kênh, ReLU để giữ ảnh hưởng dương, rồi nội suy về vùng 224x224. Heatmap được chuẩn hóa và phủ lên ảnh gốc với alpha 0,38. Việc dùng hook tạm thời và tháo hook ở finally giúp không để callback tồn tại sau request.

Grad-CAM có chi phí cao hơn phân loại thường vì cần gradient và backward pass; không nên bật mặc định cho mọi request ở máy chủ RAM thấp. Heatmap cho thấy vùng kích hoạt của lớp đã chọn trong kiến trúc, chứ không phải lời giải thích nhân quả. Có thể kiểm tra độ ổn định bằng crop/occlusion hoặc so sánh nhiều lớp; hiện báo cáo chỉ mô tả đây là công cụ minh họa giúp người dùng xem vùng ảnh tác động tới dự đoán.

6.4. AI 2 — Phát hiện vùng hoa bằng YOLO11n

ObjectDetector nạp trọng số YOLO, chuyển ảnh sang RGB và gọi predict với imgsz=320, conf=0,25, IoU=0,45, max_det=100. Mỗi box được đổi thành nhãn, confidence và tọa độ [x1,y1,x2,y2]. Kết quả plot từ Ultralytics là BGR nên code đảo kênh để tạo ảnh RGB trước khi encode và trả cho frontend. Summary đếm số vùng theo từng lớp, trong khi frontend vẽ overlay tương tác trên kích thước ảnh gốc.

Detector fine-tune cho bốn lớp nhóm hoa: daisies, dandelion, roses và sunflowers. Tập ghi nhận 831 ảnh, 1.280 bounding box; chia train/validation/test lần lượt 664/83/84 ảnh, 12 epoch ở 320 pixel. Metric test báo mAP50=0,8704; mAP50–95=0,6802; precision=0,8104; recall=0,8199. mAP50 dùng ngưỡng IoU 0,5, trong khi mAP50–95 trung bình trên nhiều ngưỡng chặt hơn nên thường thấp hơn.

Giới hạn quan trọng là detector chỉ biết các lớp gán nhãn; hoa tulip hoặc vật thể ngoài lớp có thể bị bỏ qua. Hiệu quả phụ thuộc annotation: box thiếu hoặc không nhất quán làm thay đổi precision/recall. Các ảnh có nhiều cành, hoa che khuất, chụp xa hoặc

nền phức tạp cần được bổ sung. Trong UI, box được đổi từ pixel sang phần trăm; cần xác nhận ảnh preview và hệ tọa độ server cùng kích thước/định hướng để tránh lệch khung trên ảnh xoay theo EXIF.

6.5. AI 3 — Tìm ảnh bằng ảnh truy vấn

Production Free dùng vector đặc trưng penultimate của classifier ResNet-18 để tránh giữ thêm một encoder lớn. Sau layer4, average pooling tạo vector 512 chiều; F.normalize đưa vector về độ dài đơn vị. Index lưu vector ảnh cùng metadata trong classifier_features.npy và meta.json. Với query, nhân ma trận gallery với vector query, tương đương cosine similarity khi cả hai đã chuẩn hóa, rồi sắp xếp score giảm dần để lấy top-k.

Search có thể dùng dự đoán lớp top-1 để lọc gallery nếu confidence vượt ngưỡng. Nếu caller truyền label rõ ràng, label đó được dùng thay vì dự đoán. Kiểm tra index lúc khởi tạo đảm bảo số dòng meta khớp số vector và chiều vector khớp feature dimension của lớp fully-connected; nếu lệch thì service dừng báo lỗi thay vì trả kết quả sai âm thầm. File index dùng mmap_mode='r' để tránh nạp bản sao đầy đủ khi RAM hạn chế.

6.6. AI 3 — Tìm kiếm ảnh từ mô tả tiếng Việt

Tìm kiếm văn bản khác với tìm kiếm ảnh: câu mô tả phải được biến thành biểu diễn có thể so với ảnh. Backend hỗ trợ encoder CLIP và nhánh retrieval nhẹ cho môi trường RAM hạn chế. Nhánh nhẹ chuẩn hóa Unicode, bỏ dấu trong trường hợp so khớp alias, đọc tên Việt/Anh, ý nghĩa và mô tả trong catalog, rồi khai thác cue hình thái được nhóm thiết kế. Khi truy vấn tiếng Việt không được CLIP biểu diễn tốt, query_used cho biết câu được biến đổi để truy xuất.

Bộ offline evaluation có 10 prompt thủ công trên năm nhóm hoa nổi bật. P@5 tăng từ 0,24 lên 0,32 sau khi mở rộng cue; hint chọn đúng nhóm trong 8/8 trường hợp có hint, với coverage 8/10. Con số nhỏ này chỉ là kiểm tra định hướng, chưa chứng minh mọi cách diễn đạt tiếng Việt. Cần mở rộng số người viết prompt, kiểm tra đồng nghĩa vùng miền, câu có nhiều đặc điểm và negative queries như ảnh lá, cây cảnh hoặc chủ thể không phải hoa.

6.7. AI 4 — Chatbot RAG và truy xuất nguồn

Trong ứng dụng này, retrieval lấy đoạn tài liệu liên quan trước khi chatbot trả lời. Kho hiện ghi nhận 11 nguồn tài liệu, các chunk được tách và lập chỉ mục BM25. Truy vấn được chuẩn hóa, BM25 tính điểm dựa trên tần suất token và độ hiếm tương đối; các đoạn đầu được đưa làm context. API streaming gửi danh sách nguồn trước hoặc trong luồng để giao diện hiển thị tài liệu mà câu trả lời dựa vào.

BM25 phù hợp cho tiếng Việt khi token hóa và từ khóa rõ, đồng thời chạy nhẹ mà không cần embedding model khác. Điểm yếu là khớp từ vựng không luôn nắm bắt ngữ nghĩa; cách diễn đạt khác xa tài liệu có thể xếp hạng thấp. Khi dùng sinh văn bản, context đóng vai trò ràng buộc nhưng không bảo đảm mô hình chỉ dùng nguồn hoặc không suy diễn thêm. Vì vậy, nhóm đánh giá truy xuất riêng với đánh giá chất lượng câu trả lời.

6.8. Phương án dự phòng của chatbot và chi phí

Khi Gemini được cấu hình và hoạt động, backend có thể gọi provider để diễn đạt câu trả lời trên context. Nếu thiếu key, hết quota hoặc có lỗi provider, code vẫn có câu trả lời fallback từ nguồn RAG để chức năng không phụ thuộc hoàn toàn vào API trả phí. Kiểm thử xác nhận khi không có key, endpoint chat/sync vẫn trả câu trả lời có nguồn. Gemini không cần thiết cho classifier, detector, retrieval hoặc truy xuất BM25; bật provider chỉ ảnh hưởng các luồng sinh lời văn.

Gemini là dịch vụ bên ngoài nên phụ thuộc quota, latency, điều khoản dữ liệu và cấu hình key. Mốc kiểm tra cũ trong reports cho thấy lượt gọi trả 200 ở thời điểm đó; điều này không đảm bảo khả dụng hiện tại. Dữ liệu ảnh người dùng không được chuyển cho Gemini ở luồng phân loại/detect; lời nhắn chat có thể được gửi sang provider khi bật, vì thế giao diện và chính sách riêng tư cần nói rõ. Đường fallback offline là cơ chế giảm phụ thuộc, không phải bằng chứng mọi câu trả lời đều đầy đủ.

6.9. Quản lý RAM và nạp model theo nhu cầu

api/main.py có danh sách LOADERS ánh xạ tên thành module/lớp, khóa riêng cho mỗi model, một khóa chung cho model nặng và trạng thái not_loaded/loading/ready/failed. Hàm _require nạp model khi route cần; Render Free có thể giải phóng model khác trước khi chuyển sang model mới. Cấu hình memory-aware readiness cho phép health báo service sẵn sàng khi các route được bật dù tất cả

model chưa resident; trạng thái thực tế vẫn được báo riêng để không tạo ấn tượng mô hình đã nằm trong RAM.

Chiến lược này đổi bộ nhớ lấy độ trễ: request đầu của từng model chịu thời gian nạp checkpoint và có thể khởi động nguội; request sau nhanh hơn cho tới khi model bị giải phóng. Cần kiểm tra việc thread đồng thời gọi nhiều route, trạng thái giữa lúc load, giải phóng tài nguyên CUDA/CPU và lỗi import. Việc có _inference_lock, MODEL_LOCKS và HEAVY_MODEL_LOCK phản ánh yêu cầu không để hai luồng cùng thay đổi model shared hoặc làm vượt giới hạn bộ nhớ.

CHƯƠNG 7. TÀI KHOẢN, XÁC THỰC VÀ POSTGRESQL

7.1. Mô hình dữ liệu

Bảng	Khóa và trường chính	Vai trò
users	id; email UNIQUE; display_name; password_hash; password_salt; created_at	Danh tính tài khoản
sessions	token_hash; user_id FK; expires_at	Nhiều phiên có thời hạn cho một người
user_data	(user_id, data_key) khóa ghép; payload; updated_at	Lưu garden hoặc giftbook dạng JSON

Quan hệ sessions.user_id và user_data.user_id trỏ tới users.id; xóa user làm cascade phiên và dữ liệu liên quan. Khóa chính ghép cho user_data cho phép một user có một bản ghi cho từng loại collection, cập nhật bằng upsert. Kiểu payload JSON linh hoạt giúp frontend bổ sung trường của collection mà không cần tạo một bảng con cho mọi loại phần tử; đổi lại server cần validation schema nghiệp vụ để payload sai không làm UI lỗi.

Ở local, account_store dùng SQLite và bật foreign_keys cùng WAL. Khi có DATABASE_URL, adapter dùng psycopg/PostgreSQL và chuyển dấu placeholder ? thành %s để giữ phần lớn SQL chung. Production báo cáo trước đó xác nhận PostgreSQL Neon được cấu hình trên dịch vụ Render. Tài liệu không đưa connection string, password demo hay API key vào repository; .env.example chỉ ghi tên biến và giá trị mẫu không phải bí mật.

7.2. Đăng ký, đăng nhập và băm mật khẩu

Khi đăng ký, email được strip và lowercase; backend tạo salt ngẫu nhiên 16 byte. hashlib.scrypt với n=2^14, r=8, p=1 và dklen=32 tạo password hash. Salt lưu cạnh hash để mỗi tài khoản có giá trị riêng. Khi đăng nhập, hệ thống truy vấn email không phân biệt hoa thường, chạy scrypt rồi dùng secrets.compare_digest để so sánh chống timing leak đơn giản. Nếu email không tồn tại, code vẫn chạy scrypt với salt giả để giảm chênh lệch thời gian dễ nhận biết.

Scrypt tốt hơn lưu mật khẩu thuần hoặc băm nhanh SHA-256 vì chủ động tốn tài nguyên cho mỗi lần đoán. Tuy nhiên, hệ thống chưa có đầy đủ lớp chống brute-force

như giới hạn theo IP/tài khoản, khóa tạm, xác minh email hoặc khôi phục mật khẩu. Đây là ứng dụng học tập; không nên dùng lại mật khẩu thật hay nâng trực tiếp thành dịch vụ tài khoản công cộng quy mô lớn nếu chưa bổ sung các kiểm soát này.

7.3. Cookie phiên và vòng đời token

Token phiên được tạo bằng secrets.token_urlsafe(32), trả qua cookie HttpOnly/SameSite và có thời hạn 14 ngày. CSDL chỉ lưu SHA-256 của token, nên bản ghi sessions không chứa token có thể dùng trực tiếp nếu bị đọc lộ. Khi xác minh, server hash token từ cookie rồi join sessions với users và kiểm tra expires_at. Khi logout, token hash tương ứng bị xóa khỏi database và cookie được xóa khỏi browser.

Vì cookie được gửi tự động theo request, SameSite=Lax giúp giảm một số dạng CSRF nhưng không thay thế mọi biện pháp CSRF trong tất cả luồng. Cần rà soát Secure trên HTTPS, domain/path, CORS credentials và cùng-origin rewrite. Thời hạn phiên nên được kiểm tra khi deploy; user cần logout trên máy công cộng. Một cải tiến là xoay token khi đăng nhập lại, dọn phiên hết hạn theo lịch và thêm rate limiting cho auth endpoint.

7.4. Đọc/ghi payload và cách ly dữ liệu

Route account data không nhận user_id từ body. Hàm lấy current user từ session, sau đó truyền id nội bộ cho read_user_data/write_user_data. Khóa data_key cần giới hạn vào tập cho phép như garden và giftbook để không biến endpoint thành kho JSON tùy ý. Upsert giúp cập nhật một payload nguyên tử theo khóa ghép. JSON serialize dùng ensure_ascii=False để tiếng Việt không bị chuyển thành escape khó đọc trong CSDL.

Các kiểm thử account cần chứng minh anonymous nhận 401, user A không đọc/ghi được dữ liệu user B, logout làm token cũ mất tác dụng, email trùng bị từ chối, dữ liệu tồn tại sau request mới và payload roundtrip đúng. Kiểm thử PostgreSQL thật chỉ chạy khi có TEST_DATABASE_URL; pipeline có service PostgreSQL để thực hiện kiểm tra tích hợp. Nếu một lượt test local không có DSN, trường hợp đó bị skip chứ không được tính như đã kiểm tra integration trên máy đó.

CHƯƠNG 8. CÁC CHỨC NĂNG SẢN PHẨM BỔ TRỢ

8.1. Bách khoa 103 loài hoa

Catalog data/species.json gồm id ổn định, tên Việt/Anh, mô tả, màu, mùa và thông tin phụ trợ. ID được dùng làm khóa route /bach-khoa/{id}, map nhãn classifier và tìm ảnh gallery. Endpoint /api/species trả danh sách, còn /api/species/{id}/image phục vụ ảnh tương ứng. Cấu trúc dữ liệu có thể được nạp nhanh ở client vì bộ danh mục không thay đổi theo từng người dùng.

Chất lượng trải nghiệm phụ thuộc đồng nhất ba thứ: bản ghi catalog, file ảnh cụ thể của loài và label output từ classifier. Lỗi ảnh thiếu có thể làm giao diện hiện broken image dù dữ liệu tên vẫn có. Nhóm cần có kiểm tra script để quét mọi path ảnh tồn tại, id không trùng, trường bắt buộc có giá trị và filename được chuẩn hóa. Mô tả thực vật là nội dung tham khảo; cần dẫn nguồn thực vật học đáng tin cậy nếu dùng như tài liệu khoa học.

8.2. Cây của tôi và lịch chăm sóc

gardenPlanner.js quản lý danh sách cây, loại việc chăm sóc, khoảng ngày và ngày hoàn thành gần nhất. addGardenPlant tạo mục cho speciesId; setCareInterval đổi chu kỳ theo careType; completeCare ghi mốc thời gian; nextCareDate tính lần kế tiếp; listCareTasks tổng hợp việc sắp tới. Tách logic ngày tháng khỏi component giúp kiểm thử quy tắc không cần render toàn trang.

Lịch là nhắc nhở dựa trên khoảng ngày cấu hình, không phải dự báo sinh trưởng. Nhu cầu nước/thời gian chăm sóc thay đổi theo giống, khí hậu, đất, chậu và mùa; vì vậy lời nhắc cần cho người dùng chỉnh và không nên trình bày như khuyến nghị chuẩn cho mọi vùng. Lưu ngày theo ISO giúp serialize ổn định, nhưng cần thống nhất múi giờ để việc hoàn thành sát nửa đêm không nhảy sang ngày khác khi thiết bị đổi timezone.

8.3. Sổ hoa và lịch sử nhận diện

giftBook.js chứa helper đọc, ghi và giới hạn số nhận diện gần đây. rememberRecognition thêm loài vừa nhận diện, recentUniqueRecognitions loại bản ghi trùng theo khóa và giữ số lượng gần nhất. rememberFavorite thêm/bỏ loài yêu thích. Tính năng này nối kết quả AI với hành động tiếp theo và làm cho demo có thể tiếp tục sau khi rời trang nhận diện.

Dữ liệu cục bộ tiện dụng ngay cả khi chưa đăng nhập nhưng gắn với trình duyệt hiện tại, có thể bị xóa khi clear storage và không tự đồng bộ nhiều thiết bị. Khi người dùng đăng nhập, accountSync đọc/ghi payload server. Màn hình cần thể hiện rõ nếu đồng bộ thất bại để người dùng không tưởng dữ liệu đã được lưu lâu dài. Nội dung sổ hoa không nên được xem là backup độc lập cho dữ liệu quan trọng.

8.4. Gợi ý bó hoa: quy tắc thay vì AI

gift_advisor.py dùng catalog ý nghĩa hoa, dịp, người nhận, màu và kích cỡ để tính điểm lựa chọn. Hàm recommend_gifts chuẩn hóa chuỗi, parse màu, tính khoảng cách màu, loại hoa không phù hợp dịp rồi phân bổ số cành. Ba đề xuất có thể giải thích bằng thuộc tính dữ liệu; API ghi nhãn rằng đây là gợi ý theo dữ liệu và không được xác minh. Phân biệt này quan trọng để không báo cáo sai số chức năng AI.

Quy tắc có thể kiểm tra được và chạy không cần API ngoài. Test xác nhận mọi occasion/recipient hỗ trợ trả đủ đề xuất, hoa chính được giữ khi phù hợp, loài bị cấm theo dịp không xuất hiện và dịp lạ bị từ chối. Dữ liệu ý nghĩa hoa có yếu tố văn hóa, địa phương và cá nhân; người dùng nên xem kết quả như gợi ý, không phải quy tắc ứng xử bắt buộc.

8.5. Thiệp chia sẻ và nội dung lời chúc

giftShare.js tạo payload thiệp từ loài hoa, lời nhắn, người gửi/người nhận và dịp; sau đó mã hóa token trong URL hash. Client có thể giải mã token và hiển thị thiệp mà không đưa nội dung vào query server logs thông thường. Tuy nhiên, URL vẫn có thể được sao chép hoặc lưu trong lịch sử trình duyệt, nên người dùng không nên đưa thông tin riêng tư vào lời nhắn.

giftGreetings.js cung cấp lời chúc ngoại tuyến theo bundle; khi provider bật, backend có thể sinh lời chúc cá nhân hóa rồi parser kiểm tra cấu trúc đầu ra và cache. Mọi câu do mô hình sinh có thể sáo rỗng hoặc lệch giọng, do đó fallback mẫu là cách giữ chức năng khi không có kết nối hoặc quota. Phần gợi ý bó hoa có thể chạy hoàn toàn theo luật; phần sinh ngôn ngữ là lựa chọn bổ sung và tách biệt với bốn model chính của sản phẩm.

CHƯƠNG 9. KIỂM THỬ, CI VÀ CHẤT LƯỢNG

9.1. Unit test và kiểm thử hợp đồng API

tests/test_api.py tạo FastAPI TestClient và thay mô hình bằng FakeClassifier/FakeDetector/FakeSearch/FakeBot. Cách thay thế này làm test nhanh, lặp lại được và không cần GPU hay checkpoint lớn. Các test bao phủ phản hồi health, startup, upload hợp lệ, file hỏng, query trống, k không hợp lệ, nhãn lọc, gallery, chat sync/SSE, fallback khi không có key và gợi ý bó hoa.

Fake model xác nhận hợp đồng giữa route và model interface, nhưng không đo chất lượng model thật. Vì vậy dự án cần cả unit/API tests lẫn model evaluation và production smoke. Một test giả có thể trả đúng shape dù model thật bị hỏng khi load; ngược lại, accuracy tốt không chứng minh endpoint xử lý validation đúng. Hai lớp kiểm tra trả lời hai câu hỏi khác nhau và phải được trình bày riêng.

9.2. Kiểm thử xác thực và PostgreSQL

tests/test_account_api.py kiểm tra đăng ký, email trùng, login sai, phiên, logout, anonymous access, lỗi backend được ẩn và roundtrip dữ liệu cá nhân. Kiểm thử PostgreSQL thật dùng biến TEST_DATABASE_URL; nếu biến không tồn tại, test được skip kèm thông báo. Pipeline được cấu hình service PostgreSQL để có thể thực hiện phần tích hợp trên CI, tránh phụ thuộc CSDL máy cá nhân.

Một bản tóm tắt CI trước đó ghi nhận 107 test backend/API đạt, 24 frontend test đạt và PostgreSQL integration ở pipeline. Các con số là snapshot gắn với run cụ thể, không tự động đại diện cho mọi commit sau đó. Khi chuẩn bị báo cáo, cần trỏ vào run ID lưu trong evidence và xác nhận trạng thái check của commit giao nộp cuối cùng; số test có thể tăng/giảm khi code đổi.

9.3. Kiểm thử frontend và trạng thái lỗi

Các module như accountSync, giftBook, giftFlow, giftShare, giftGreetings và gardenPlanner có test logic riêng. Đơn vị nhỏ cho phép xác nhận merge không trùng, tạo/giải mã URL, lọc dịp, ngày chăm sóc kế tiếp và lựa chọn hoa mà không phải mô phỏng cả ứng dụng. Component tests hoặc build xác nhận cú pháp JSX, liên kết module và đóng gói tài nguyên.

Những ca cần xem trong giao diện gồm: người dùng hủy upload, ảnh không hợp lệ, mạng timeout, server 503 do khởi động model, không phát hiện object, truy vấn không có kết quả, chưa login mà mở dữ liệu cá nhân và mất mạng giữa khi đồng bộ. Trạng thái rỗng cũng quan trọng: nó giúp phân biệt "đã xử lý nhưng không tìm thấy" với "request chưa chạy". Demo tốt cần trình diễn ít nhất một đường thành công và một lỗi được xử lý.

9.4. CI, build và quét bảo mật

.github/workflows/test.yml tự động chạy các job backend, frontend và kiểm tra bảo mật theo trigger repository. Nhóm kiểm tra backend test, frontend build/test và audit dependency; pipeline PostgreSQL hỗ trợ kiểm thử tích hợp. Các tệp cấu hình workflow là bằng chứng quy trình; để chứng minh commit hiện tại xanh cần xem check run chính xác thay vì ảnh của một commit cũ.

CI giảm khả năng merge thay đổi làm gãy chức năng đã có, nhưng không kiểm tra mọi yếu tố như hiệu năng GPU, ảnh ngoài miền, deployment secret thực tế hoặc lỗi giao diện trên thiết bị vật lý. Security audit dependency tìm phiên bản package có advisory đã biết; nó không chứng minh code không có lỗ hổng. Cần kết hợp kiểm tra thủ công, kiểm soát quyền truy cập, rà soát dependency và theo dõi log sau deploy.

9.5. Phân tầng bằng chứng kiểm thử

Báo cáo phân biệt bốn tầng. Unit test kiểm tra hàm cô lập. API test dùng fake model kiểm tra schema/route. Integration test có PostgreSQL xác nhận adapter và ràng buộc dữ liệu. Production smoke gọi endpoint thật để xác nhận đường truyền, deploy và model được nạp. Chỉ tầng cuối mới cho biết một endpoint thật trả lời ở thời điểm kiểm tra; nhưng một request production không đủ để suy ra SLA hay accuracy tổng quát.

Với AI, benchmark dùng tập query/ảnh đã xác định và phép đo chuyên biệt. Với auth, cần tài khoản test và kiểm tra quyền trước/sau login. Với vận hành, health và startup chỉ xác nhận trạng thái service theo định nghĩa của chúng. Cách phân tầng này ngăn việc cộng mọi test thành một con số rồi tuyên bố hệ thống "đã được kiểm chứng toàn diện".

CHƯƠNG 10. TRIỂN KHAI VÀ VẬN HÀNH

10.1. Phát triển cục bộ bằng Docker Compose

compose.yaml khai báo một dịch vụ vuon-hoa, build từ Dockerfile.compose, map cổng 7860 và gắn named volume vuonhoa-data cho SQLite local. Environment đặt APP_DB_PATH, bật chế độ memory-aware, dùng retrieval backend classifier và tắt Gemini mặc định. Lệnh khởi động trong README là docker compose -f compose.yaml up --build. Nếu checkpoint quản lý bằng Git LFS chưa có trong checkout, cần kéo file LFS trước build.

Compose gom các bước dựng backend và cấu hình môi trường vào một lệnh, giúp giảng viên dễ tái tạo hơn. Nó không thay thế cấu hình cloud: Vercel và Render có biến môi trường, rewrite, cổng và persistent storage riêng. Named volume giúp giữ SQLite khi container được tạo lại trên cùng máy; xóa volume sẽ xóa dữ liệu đó. Cần phân biệt dữ liệu local trong volume với Neon production để không suy luận nhầm hai môi trường cùng kho dữ liệu.

10.2. Frontend Vercel và API Render

Frontend được triển khai trên Vercel, API container trên Render. Rewrite cùng origin cho phép browser dùng đường /api, sau đó nền tảng chuyển tới service phù hợp. Render service bind cổng từ biến PORT; health endpoint hỗ trợ nền tảng và người vận hành kiểm tra. Gói Free có thể ngủ khi không hoạt động, tạo độ trễ khởi động nguội và giới hạn CPU/RAM; do đó smoke latency lần đầu không thể so trực tiếp với máy chạy model đã warm.

Lợi ích của tách hosting là Vercel phục vụ static assets qua CDN, Render chạy môi trường Python nặng hơn. Chi phí là phải đồng bộ branch/revision, environment variable, timeout và logs. Mỗi lần sửa secret cần redeploy để process mới đọc giá trị. Trước buổi bảo vệ, cần mở trang deploy để xác nhận trạng thái Live, kiểm tra endpoint trên mạng trường và không dựa vào một ảnh chụp cũ.

10.3. Readiness, health và quan sát lỗi

GET /api/health trả trạng thái tổng quan cùng revision và trạng thái từng model. GET /api/startup được dùng như startup probe; trong chế độ Render Free, readiness có thể true dù model chưa resident vì chúng được nạp lazy. Vì vậy, model_states và chỉ số

resident phải được đọc riêng với ready. Health xanh chứng minh tiến trình API phục vụ được; chỉ smoke inference mới chứng minh route model xử lý ảnh hoặc query tại thời điểm đó.

Middleware ghi X-Process-Time-ms và log API_PERF cho route nặng, kèm RSS nếu có. Các log này giúp so sánh độ trễ, nhận diện lỗi cold start và mức tăng RAM. Log không nên chứa API key, mật khẩu, cookie, ảnh gốc hoặc nội dung tài khoản. Khi debug lỗi production cần dùng request id/route/status/thời gian và giữ lại phần chi tiết nhạy cảm ở nơi hạn chế quyền.

10.4. Kịch bản vận hành khi demo

Trước buổi trình bày, nhóm kiểm tra frontend URL, API health, login demo, truy vấn catalog, ảnh mẫu có sẵn, trạng thái Compose và file video dự phòng. Do Render Free có thể cold start, hãy gọi health sớm và gửi một yêu cầu inference trước khi đến phần demo để model nạp trước. Nếu yêu cầu đầu chậm, giải thích đó là hạn chế gói free và chờ kết quả thay vì bấm lặp liên tục tạo tải.

Nếu server không phản hồi, chuyển sang video 3–5 phút đã chuẩn bị và giải thích phạm vi của video. Video là phương án dự phòng, không chứng minh deploy live đang hoạt động. Nếu đăng nhập lỗi, không đọc credential trên máy chiếu công khai; nhập từ một bản riêng và sau buổi bảo vệ nên đổi/xóa mật khẩu demo. Các bước này vừa giữ mạch trình bày vừa tránh công khai bí mật thật.

CHƯƠNG 11. ĐÁNH GIÁ, GIỚI HẠN VÀ HƯỚNG PHÁT TRIỂN

11.1. Kết quả benchmark tổng hợp

Tác vụ	Dữ liệu / cách đo	Kết quả đã ghi nhận
Classifier ResNet-18	11.859 ảnh, 103 lớp; validation 2.135 ảnh	accuracy 90,26%; macro-F1 91,56%; weighted-F1 90,24%
Detector YOLO11n	84 ảnh test, 4 lớp	mAP50 0,8704; mAP50–95 0,6802; precision 0,8104; recall 0,8199
Image retrieval ResNet feature	1.236 ảnh index; 206 query held-out theo path	P@5 0,9165 sau lọc lớp; ranking global 0,8757
MobileCLIP2-S0 benchmark	cùng gallery/query	P@5 0,8592; không phải backend production Free
RAG BM25	31 câu hỏi, 11 nguồn	Hit@3 31/31; fallback có câu trả lời và đúng nguồn mong đợi 31/31
Vietnamese text search fallback	10 prompt thủ công	P@5 0,32; cue đúng nhóm 8/8 trường hợp có hint

Kết quả được lấy từ các tệp JSON/CSV reports, không gộp thành một điểm chung. Thông tin về tập đánh giá, seed và cách chia là một phần của metric; nếu thiếu, người đọc không thể so sánh kết quả. Một metric cao trên một bộ nhỏ có thể do gallery/query cùng nguồn ảnh hoặc câu hỏi do chính nhóm viết. Bởi vậy, các kết quả được trình bày như bằng chứng kỹ thuật cho prototype, chưa phải chứng nhận chất lượng thương mại.

11.2. Phân tích lỗi và nguyên nhân có thể

Classifier có thể nhầm những loài có hình thái gần nhau, nhất là khi ảnh chụp một phần, màu lạ hoặc không có bông rõ. Detector có thể bỏ sót hoa nhỏ và box chồng lấn; ngưỡng conf làm tradeoff giữa false positive và recall. Retrieval dựa trên feature classifier có thể ưu tiên ảnh có màu/bố cục giống nhau nhưng khác loài; lọc theo nhãn giúp giảm trường hợp đó nhưng tăng rủi ro mất kết quả đúng khi top-1 sai.

Text search chịu ảnh hưởng của từ khóa và cue được thiết kế. Chatbot có thể truy xuất sai nguồn khi câu hỏi ngắn hoặc không có từ khóa trùng; mô hình sinh có thể bổ sung thông tin ngoài context. Các dạng lỗi này cần log ẩn danh, bộ câu hỏi chuẩn, phản hồi người dùng và phân tích confusion matrix theo lớp. Báo cáo cần nêu một số trường

hợp lỗi thực tế thay vì chỉ trình bày trung bình, nhưng không nên đưa ảnh người dùng nếu chưa có quyền sử dụng.

11.3. Rủi ro, giới hạn và trạng thái cần theo dõi

Tập classifier chưa được xác nhận bằng tập test độc lập lớn; detector chỉ có bốn lớp; text retrieval có benchmark nhỏ; RAG có 11 nguồn và Hit@3 không đo đúng/sai toàn bộ câu trả lời. Gói Render Free có thể cold start hoặc hết RAM. Thời gian production ghi nhận từ một ảnh mẫu không phải cam kết response time. Đồng bộ tài khoản dựa trên payload nguyên khối nên xung đột đa thiết bị chưa có quy trình giải quyết đầy đủ.

Auth hiện có scrypt, cookie HttpOnly và kiểm tra quyền theo user, nhưng cần rate limit, email verification, reset password, policy CSRF rõ hơn và diễn tập backup. Frontend có biện pháp giảm ảnh, nhưng cần xác minh giới hạn upload tại mọi lớp. Gemini/quota và điều khoản dịch vụ có thể thay đổi. Những giới hạn này nên được nói rõ khi bảo vệ; thừa nhận phạm vi chưa làm là một phần của đánh giá kỹ thuật nghiêm túc.

11.4. Lộ trình cải tiến ưu tiên

Ưu tiên thứ nhất là tạo test set độc lập, khử trùng lặp ảnh theo perceptual hash và công bố kết quả theo từng lớp. Thứ hai là mở rộng detector với lớp hoa được người dùng tìm nhiều, tăng ảnh che khuất và kích cỡ khác nhau. Thứ ba là tăng bộ prompt tiếng Việt có câu âm tính, so sánh CLIP đa ngôn ngữ với phương án fallback và đánh giá MRR/P@k từ người gán nhãn độc lập.

Với RAG, bổ sung nguồn có tác giả/ngày, đánh giá recall nguồn, factuality và câu trả lời từ ít nhất hai người chấm. Với account, thêm rate limiting, khôi phục tài khoản, xác nhận email, version payload, conflict warning và backup-restore drill. Với vận hành, dùng health alert, đo p50/p95 theo nhóm request, giới hạn concurrency và xem xét gói tài nguyên phù hợp. Mỗi cải tiến cần có tiêu chí chấp nhận trước khi code để tránh chỉ tối ưu cảm giác.

CHƯƠNG 12. HƯỚNG DẪN ĐỌC CODE VÀ KỊCH BẢN BẢO VỆ

12.1. Đường đi từ UI tới API

Khi giảng viên chọn file trong Classify, hãy lần theo Classify.jsx → postImage trong api.js → POST /api/classify trong api/main.py → _require('classifier') → ImageClassifier.predict trong core/classifier.py → JSON response → state.status='ok'. Đây là đường đọc có thể kiểm chứng bằng code và phù hợp để trình bày cách React hooks, REST API và model service phối hợp.

Các câu hỏi nên tự trả lời: vì sao upload là multipart; tại sao model không được tạo lại mỗi request; status nào client dùng để thử lại; confidence thấp được hiển thị ra sao; Grad-CAM chạy khác inference thường thế nào. Khi trình bày code, phóng đủ lớn theo yêu cầu slide và chỉ đưa một đoạn ngắn có thể đọc từ cuối phòng. Những chi tiết implementation còn lại để ở báo cáo thay vì nhồi chữ lên slide.

12.2. Đường đi từ login tới bản ghi CSDL

Route register/login nằm trong api/main.py, hàm lưu trữ nằm trong api/account_store.py. Hãy giải thích: email normalize để tránh tạo nhiều account do hoa/thường; salt ngẫu nhiên trước khi chạy scrypt; token raw chỉ trả cho client qua cookie, database giữ hash; get_user join session và user; route dữ liệu lấy user_id từ phiên chứ không tin body. Đó là các bước thể hiện xác thực thực sự chứ không phải một cờ login chỉ tồn tại trong React.

Khi user lưu Cây của tôi, client serialize collection qua PUT /api/account/data/garden; server lấy current account và upsert theo (user_id, data_key). Thử gọi GET khi chưa login phải nhận 401. Đây là demo lỗi ngắn và trực quan về kiểm soát quyền. Không hiển thị DATABASE_URL hoặc API key khi mở trang cấu hình; biến môi trường chỉ được đọc tại runtime.

12.3. Đường đi retrieval và chatbot

Để giải thích tìm ảnh, bắt đầu tại Search.jsx: text form gọi postJson còn image picker gọi postImage. API route gọi engine, engine chuẩn hóa query hoặc trích feature rồi rank gallery. UI chỉ cần nhận results, query_used và score để hiển thị. Điểm similarity không được ghi nhãn là xác suất đúng. Câu hỏi hay là điều gì xảy ra nếu model phân

loại top-1 sai nhưng ảnh cùng loài vẫn có trong gallery; câu trả lời là filter có thể giảm recall và cần benchmark global ranking.

Với chatbot, giải thích ba lớp: retrieve đoạn văn, tạo câu trả lời có context, gửi sources/token qua SSE. Khi provider ngoài không khả dụng, fallback vẫn trả nội dung trích dẫn từ nguồn. Hãy phân biệt metric Hit@3 (tìm đúng tài liệu) với đánh giá factuality (câu trả lời đúng) và nêu rằng báo cáo hiện có phép đo thứ nhất nhưng chưa có đánh giá người chấm đầy đủ cho phép đo thứ hai.

12.4. Bộ câu hỏi tự kiểm tra trước khi bảo vệ

Mỗi thành viên nên có thể đọc code một route, một test và một quyết định thiết kế thuộc phần đã làm. Câu hỏi mẫu: vì sao React dùng useEffect cleanup; vì sao file ảnh phải mở thử ở backend; tại sao model lock cần thiết; foreign key cascade giải quyết việc gì; HTTP 401 khác 403 thế nào; P@5 khác mAP ra sao; nếu Render ngủ thì người dùng thấy gì; Compose volume giữ dữ liệu nào; tại sao key bị thiếu không được thay bằng chuỗi giả trong production.

Một cách luyện hiệu quả là chọn ngẫu nhiên hàm trong repo, giải thích đầu vào/đầu ra, trạng thái lỗi, tác động lên dữ liệu và một test bao phủ. Nếu không giải thích được, đọc hàm gọi nó và test trước, sau đó chạy sửa nhỏ trong nhánh thử. Không nên học thuộc báo cáo mà không thể lần từ mô tả tới file code. Giảng viên có thể hỏi ngược một điều kiện biên chưa nằm trong demo, nên hiểu giả định quan trọng hơn nhớ nguyên văn.

12.5. Phân công công việc và trách nhiệm nhóm

Thành viên	Vai trò	Phần việc được phân công	Tỷ lệ tự đánh giá
Nguyễn Hữu Lê Hiếu — 23010895	Trưởng nhóm	Tích hợp sản phẩm, kiến trúc, API/AI và điều phối kiểm thử	34%
Phạm Văn Huấn — 23010331	Thành viên	Giao diện và trải nghiệm người dùng, nội dung/catalog	33%
Đào Duy Đông — 24100085	Thành viên	Tài khoản, dữ liệu, kiểm thử và hỗ trợ triển khai	33%

Tỷ lệ trên là bản tự đánh giá nhóm đã thống nhất để cân bằng vai trò; mỗi người cần rà lại phần việc thực tế và ký xác nhận trên bảng phân công riêng trước khi nộp. Bảng này không thay cho lịch sử commit cá nhân theo yêu cầu repo. Mỗi thành viên chịu trách nhiệm giải thích phần mình khai và cần thực hiện commit từ đúng tài khoản GitHub cá nhân theo quy định môn học.

12.6. Khai báo sử dụng công cụ AI

Nhóm sử dụng OpenAI Codex/ChatGPT trong quá trình đọc cấu trúc repository, gợi ý hướng truy vết lỗi, hỗ trợ soạn/chỉnh sửa code và chuẩn bị tài liệu kỹ thuật. Một số nội dung báo cáo được hỗ trợ biên tập; các mô tả chức năng và số liệu phải được đối chiếu với source, test hoặc evidence trong repository. Công cụ không thay thế trách nhiệm kiểm chứng, review và giải thích của thành viên.

Các phép đo mô hình được dẫn tới artifact và script đánh giá trong reports/ và scripts/. Cần cập nhật khai báo này nếu nhóm còn sử dụng thêm công cụ AI khác như Copilot, Gemini hoặc Claude ở phần code/tài liệu; báo cáo không tự khẳng định những công cụ không có bằng chứng. Quy định môn học cho phép dùng AI để học và tìm lỗi nhưng yêu cầu khai báo công cụ và phần việc; từng sinh viên chịu trách nhiệm với mọi dòng code mình nộp.

KẾT LUẬN

Vườn Hoa AI kết hợp tra cứu catalog, các luồng AI, quản lý dữ liệu tài khoản và một số tiện ích chăm sóc/tặng hoa trong cùng giao diện. Giá trị chính của đồ án là thể hiện được hành trình từ giao diện React tới API, model runtime và PostgreSQL, đồng thời xây dựng kiểm thử cho hợp đồng route, tài khoản và các quy tắc nghiệp vụ. Bốn nhóm tác vụ AI được đo riêng theo metric phù hợp, kèm giới hạn của dữ liệu và cách thử.

Sản phẩm có thể dùng làm prototype học tập và demo; bằng chứng hiện có chưa đủ để tuyên bố độ chính xác tổng quát, khả năng chịu tải cao hoặc chuẩn bảo mật của dịch vụ thương mại. Trước khi nộp, nhóm cần kiểm tra lại deploy đang Live, cập nhật commit/tag cuối, mở video dự phòng, hoàn thiện chữ ký bảng phân công và xác nhận các thành viên hiểu phần mình phụ trách. Tài liệu được xem là mô tả kỹ thuật tại ngày 26/10/2026; trạng thái dịch vụ ngoài có thể đổi sau ngày này.

NGUỒN KIỂM CHỨNG TRONG REPOSITORY

1. Giao diện và API client: web/src/App.jsx, web/src/api.js, web/src/features/.
2. Routes FastAPI: api/main.py; store tài khoản: api/account_store.py.
3. Mô hình: core/classifier.py, core/detector.py, core/retrieval.py, core/retrieval_light.py, core/llm.py.
4. Gợi ý tặng hoa: core/gift_advisor.py; dữ liệu: data/species.json và data/.
5. Kiểm thử: tests/, web/src/*.test.js và .github/workflows/test.yml.
6. Benchmark: reports/classifier_evaluation.json, reports/detector_evaluation.json, reports/classifier_retrieval_precision_at_5.json, reports/rag_hit_at_3.json và reports/vietnamese_search_offline_precision_at5.json.
7. Production smoke và Compose: reports/current-production-*.json, reports/current-clean-checkout-compose-ai-smoke-2026-10-23.json, compose.yaml, Dockerfile.compose và README.md.

PHỤ LỤC A. MỘT SỐ ĐOẠN CODE CẦN NẮM

A.1. Khóa suy luận và đường nạp model

```
LOADERS                                          =                                          {
    "classifier":               ("core.classifier",               "ImageClassifier"),
    "detector":                  ("core.detector",                  "ObjectDetector"),
    "retrieval":                   ("core.retrieval",                   "ImageSearch"),
    "llm":                         ("core.llm",                         "RAGChatbot"),
}
MODEL_LOCKS     =     {name:     threading.Lock()     for     name     in     LOADERS}
HEAVY_MODEL_LOCK = threading.RLock()
```

Danh sách loader là registry để route không phải hard-code import ở nhiều nơi. Khóa model điều phối hai request cùng muốn nạp một model; khóa nặng giới hạn các tác vụ có footprint RAM lớn. Nếu bỏ khóa, hai request đồng thời có thể cùng load checkpoint hoặc giải phóng model trong khi request khác đang dùng. Hãy mở thêm hàm _require và _evict_for_render_free để theo dõi toàn bộ vòng đời.

A.2. Token phiên không lưu trực tiếp

```
token                            =                            secrets.token_urlsafe(32)
token_hash                =                hashlib.sha256(token.encode()).hexdigest()
db.execute(
    "INSERT    INTO    sessions(token_hash,    user_id,    expires_at)    VALUES(?,?,?)",
    (token_hash,                          user_id,                          expires_at),
)
return token
```

Khách nhận token để browser gửi cookie ở request sau; store chỉ giữ hash. Khi xác minh, server hash token nhận được và truy vấn theo token_hash. Mẫu này làm giảm tác động khi bảng sessions bị lộ, nhưng không bảo vệ nếu kẻ tấn công đã lấy được cookie raw từ thiết bị hoặc log. Bởi vậy, HttpOnly, HTTPS, không ghi cookie vào log và logout xóa session vẫn cần thiết.

A.3. Dùng cùng model để phân loại và truy xuất

```
features                              =                              model.layer4(features)
features                     =                     model.avgpool(features).flatten(1)
probs                  =                 model.fc(features).float().softmax(dim=-1)[0]
vector = F.normalize(features.float(), p=2, dim=1)[0]
```

Phần backbone dùng chung tạo embedding và phần fully connected tạo logits phân loại. Đây là cách tiết kiệm RAM vì không nạp thêm encoder chỉ để tìm ảnh. Hai mục

tiêu huấn luyện khác nhau: classifier tối ưu nhãn lớp, còn retrieval cần vector gần nhau khi ảnh cùng ngữ nghĩa. Chia sẻ representation giảm tài nguyên nhưng có thể làm truy xuất kém tinh tế hơn mô hình metric-learning chuyên dụng.

A.4. Kiểm tra payload và cách ly theo user

```
def read_user_data(user_id: int, key: str):
    with _connect() as db:
        row = db.execute(
            "SELECT payload FROM user_data WHERE user_id=? AND data_key=?",
            (user_id, key),
        ).fetchone()
    return json.loads(row["payload"]) if row else None
```

Điểm phân quyền nằm ở tham số user_id mà route lấy từ current session, không phải từ JSON của browser. Hàm store nên được gọi sau validation data_key và payload. Người đọc code nên lần call site trong api/main.py để xác minh route không cho phép client tự chọn user_id. Test cách ly hai tài khoản là bằng chứng quan trọng hơn việc chỉ nhìn thấy foreign key trong schema.

A.5. Tài liệu đánh giá cần giữ kèm

Các JSON/CSV báo cáo lưu cấu hình và metric; scripts/ giữ chương trình tái tạo một phần đánh giá. Khi đổi checkpoint hoặc index, nhóm cần chạy lại script, cập nhật file kết quả, ghi commit/hash artifact và thay con số trong báo cáo. Không sửa tay metric trong PDF mà không đồng bộ file máy đọc được. Nếu dataset không thể công khai do license/kích thước, tối thiểu ghi phiên bản, nguồn, split, seed và cách lấy mẫu để phép đo có thể được xem xét.